Do not write in this margin

Centre No: Centre Name:

Code: Subject: Computer Science SL

Candidate Number: 4003 Candidate Name: GUANYI ZHOU

Signature: 周观颐

Do not write in this margin

1. (a) global variables can be used everywhere while local variables aren't allowed to use in other places. Local variables are more convenient to be corrected or ~~replaced~~ debug.

(b) 9.4
2
9.4

(c)

a	a<20?	b
2	true	0
~~6~~ 6	true	2
18	true	8
54	false	26

(d)

2. (a) ==
====
done
three lines

(b) public static int smallest (int a, int b, int c){
~~int min = 0;~~ if (a<b && a<c) {
~~if (a > max) {~~ return a; }
else if (b<a && b<c){
return b; }
else if (c<a && c<b){
return c; }

Evelyn — 29/40 (Q1 10/12 · Q2 10/15 · Q3 9/13)

Q1(d) 0/2
Not answered. A do...while tests its condition AFTER the body, so the body always runs once: it prints 3.

Q2(b) 3/4
Your pairwise tests find the smallest of three DIFFERENT values, but if two values are equal no branch runs and the method prints instead of returning - every path must return a value. Standard form: int min = a; if (b < min) min = b; if (c < min) min = c; return min;

Do not write in this margin

```
else {
System.out.println("There are same values.");}
}
```

(c)(i) 4

(ii) It's more convenient to use a method from a library and more readable. Also, it cost fewer bytes.

(d)(i) 1 to 8 inclusive

(ii)
```
public static int roll() {
    int num = rnd.nextInt(8);
    return num;}
```

3. (a) Because the array is more convenient to use and cost fewer bytes. Each Students' marks are same type variables, so it's better to use an array to hold them together.

(b) 3
243

(c)
```
public static double average(int[] nums){
    int[] nums = {45, 62, 71, 58, 90, 33};
    int avr = 0;
    int sum = 0;
    for (int i = 0; i < nums.length; i++) {
        sum = sum + nums[i]; }
    avr = sum / nums.length;
    return avr; }
```

(d)
```
int [][] 2D_array. seats {{1,2,3},{4,5,6},{7,8,9},
    {10,11,12}};
System.out.print (seats[2][3]);
```

Q2(c)(i) 0/2
Math.abs(-6) is 6 and Math.max(6, 4) is 6, so the output is 6, not 4.

Q2(c)(ii) 1/2
'More convenient / fewer bytes' is close to reuse of code. Add the second standard advantage: library methods are already tested, so they improve reliability.

Q2(d)(i) 1/2
The SIZE of your range is right but it is shifted by one: nextInt(8) returns 0 to 7 - the argument 8 is the upper bound and is NOT included. (-1)

Q2(d)(ii) 2/2
Follow-through (FT): given your answer to (d)(i), adding 1 is the consistent fix, so no marks are lost here. For (d)(i) the correct range is 0-7, so the method is: return rnd.nextInt(8) + 1;

Q3(a) 2/3
Same type + convenient are two good points; add that the values can then be processed together in a LOOP using the index (for example adding them all up).

Q3(b) 2/3
count = 3 is right, but the total is 62 + 71 + 90 = 223, not 243.

Q3(c) 4/4
Correct. One note: avg is an int, so sum / nums.length truncates the fraction. Declare double avg (or cast (double) sum / nums.length).

Q3(d)(i) 1/2
int[][] is right, but you created a 4 x 3 array of literals. The question wants 3 rows and 4 columns: int[][] seats = new int[3][4];

Q3(d)(ii) 0/1
Indices start at 0, so row 2, column 3 is seats[1][2], not seats[2][3].